



SABANCI UNIVERSITY

EE310 - HARDWARE DESCRIPTION LANGUAGES

# Eyeriss-Inspired CNN Accelerator

Final Project Report

**Students:** Ahmet Poyraz Güler, Mete Sakallı

**ID:** 30971, 30670

**Instructor:** Ozcan Ozturk

# 1 Overview of Building Blocks

Our accelerator is inspired by the Eyeriss architecture [1], which prioritizes energy efficiency and reconfigurability for deep convolutional neural networks (CNNs). This section presents an overview of the major components implemented in our FPGA prototype, with emphasis on architectural modifications and design decisions that differentiate it from the original chip.

## 1.1 Processing Element (PE) Grid

The PE grid forms the computational core of our accelerator. Similar to Eyeriss, our design uses a spatial array of PEs arranged in a  $12 \times 14$  grid, enabling parallel convolution operations across multiple channels and spatial locations.

Each PE performs a Multiply-Accumulate (MAC) operation on 16-bit fixed-point (Q7.8) image and filter values. Unlike the original Eyeriss design, which includes dynamic clock gating and local scratchpads for fine-grained reuse and power optimization, our implementation omits these features for simplicity and synthesis feasibility on our target FPGA. One key constraint while working with FPGA's was our inability to incorporate local scratchpads.

### Differences from Eyeriss:

- No fine-grained data gating or zero-skipping inside PEs.
- No local scratchpad memories; all data reuse is handled via external buffering.
- Fixed 3-stage pipelined PE with deterministic scheduling.

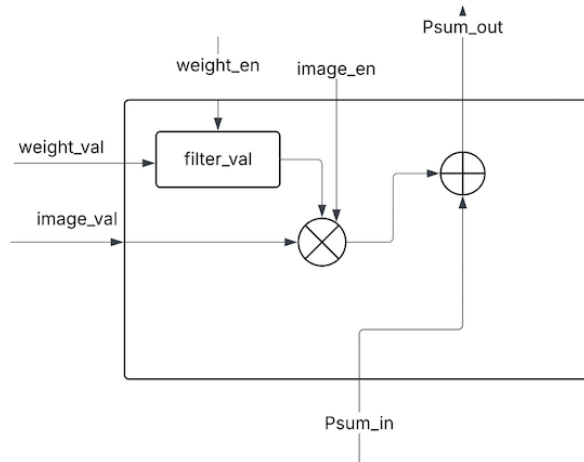


Figure 1: PE Structure and Dataflow

## 1.2 Global Buffer

Our global buffer mimics the 108KB SRAM buffer in Eyeriss by using registers on the FPGA. It stores image tiles, filter weights, and partial sums for reuse across passes.

In contrast to the hierarchical memory and tiling strategy of the original design, we use a single-layer buffer with dual-port access. The buffering logic is static and managed by the FSM, rather than being dynamically reconfigured by software.

### Differences from Eyeriss:

- No dynamic tiling or runtime buffer reconfiguration.
- Static size and fixed mapping to BRAM.

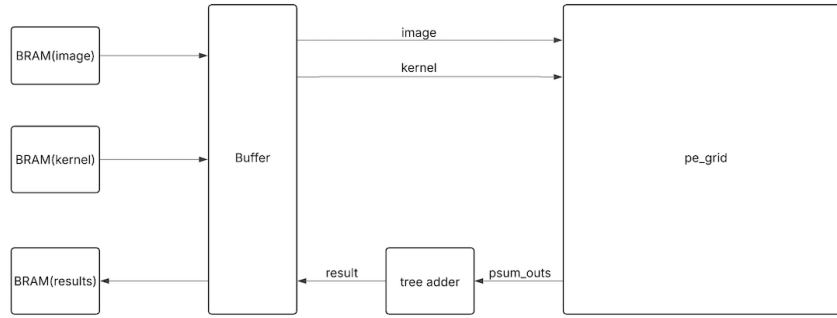


Figure 2: Global Buffer Organization and Dataflow

## 1.3 Router

Eyeriss features a flexible multicast NoC with row-wise and column-wise buses allowing for dynamic data delivery based on tagged IDs. Our implementation simplifies this by adopting a row-wide multicast router, where a tag signal enables selective PE activation within a row.

### Differences from Eyeriss:

- No column buses; only horizontal (row-wide) multicasting.
- No configurable ID-based routing; simple tag matching.
- Reduced NoC power and complexity at the cost of reduced routing flexibility.

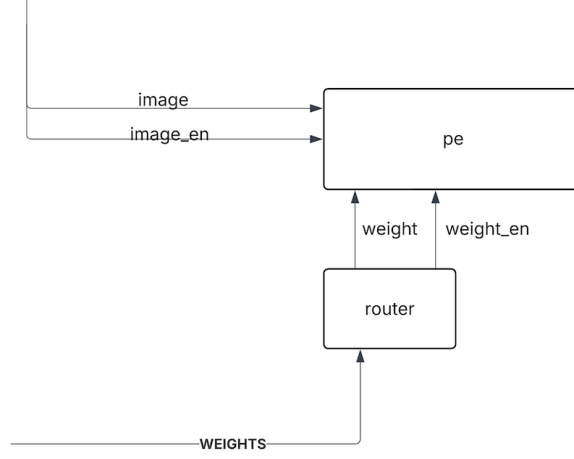


Figure 3: id matching router logic

#### 1.4 Router Network (Multicast NoC)

Our design implements a simplified version of the flexible multicast NoC proposed in Eyeriss. While the original Eyeriss design uses a combination of global buses and tag-based addressable multicast to any PE in the 2D grid, our implementation divides the responsibilities based on data type to minimize complexity and improve efficiency.

Each PE is assigned a `tag_row` value, and multicast data is broadcast horizontally across an entire row. Only the PEs whose row ID matches the tag latch the data. This tag-based row-wise multicast is used specifically for distributing **filter values**.

In contrast, **image values** are distributed through direct global fanout wires connected to all PEs in a column. There is no tag matching involved for image injection—every PE receives the same image value at once, emulating global broadcasting without routing logic. This further reduces dynamic routing complexity while ensuring image reuse across vertical computation paths.

##### Our Multicasting Strategy:

- **Row-wise Multicast (for Filters):** Each filter value is broadcast across a row. Only PEs with matching `tag_row` accept the value.
- **Global Multicast (for Image Values):** Image values are hardwired to all vertical columns without tag logic. Every PE receives the value simultaneously.

##### Comparison with Eyeriss:

- **Our Row-wise Multicast:** Simpler routing, less control logic, suitable for predictable filter distribution.
- **Eyeriss Global Multicast:** Uses configurable address-based multicast over shared NoC, supporting arbitrary routing but at higher cost.

Figure 4 illustrates a two-cycle row-wise multicast operation for filter values. Over multiple cycles, different rows are activated using tag matching. The rightmost diagram shows a conceptual global broadcast for image values via direct vertical injection.

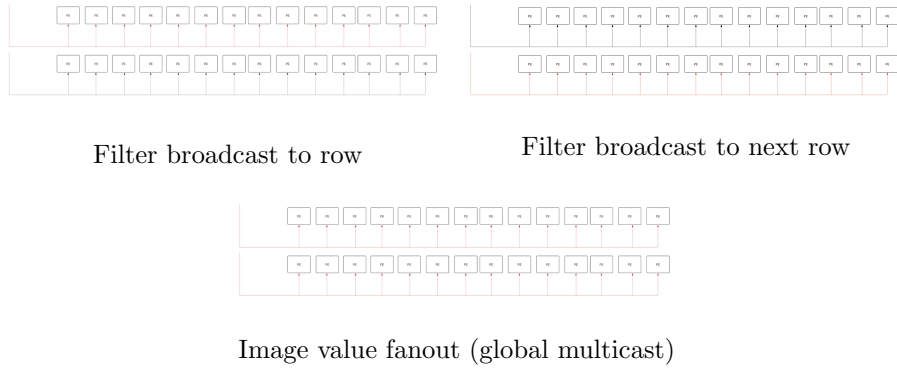


Figure 4: Multicasting strategies: row-wise tag matching (left, center), and global fanout for image data (right)

## 1.5 Adder Tree

We use a fully combinational 14-input adder tree to reduce partial sums generated by each PE row. This differs from Eyeriss, which stores and accumulates psums via scratchpads and optionally compresses them before offloading to DRAM. Our implementation focuses on producing single results for single convolutions

### Differences from Eyeriss:

- Sums are immediately available as 16-bit fixed-point values.

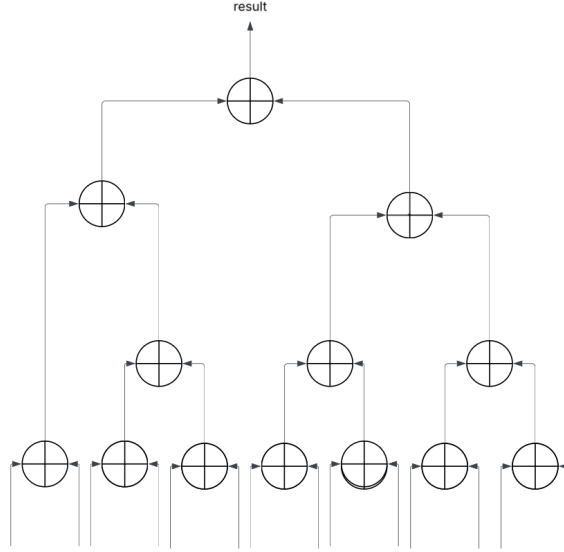


Figure 5: Adder Tree Architecture

## 1.6 FSM-based Controller

Instead of the programmable scheduling logic in Eyeriss, we designed a finite state machine (FSM) to manage control signals for image/weight injection, tag distribution, and result collection. It includes states for idle, load, compute, and finalize.

### Differences from Eyeriss:

- FSM state transitions are static and clock-driven.
- No dynamic configuration or run-time software support.
- Limited flexibility but simpler for synthesis and debugging.

In the FSM we've written, there are a total of 15 states. These states follow the path of:

- Loading data from BRAM (kernel and image),
- Sending data to the PEs and obtaining the partial sum,
- Displaying the results on hardware.

Loading data from BRAM requires planning. Since the PE grid cannot import large sets of data all at once, we've decided to partition data accordingly, both while designing the BRAM coefficient files, and while importing the data from the BRAM.

We've decided to take 4x4 inputs from the image, left-to-right, top-to-bottom, as the kernel we tested is also 4x4. In a nutshell, our stride is 4. To do this, we've created a helper python function that takes the whole input image matrix (2D), and rearranges it to the `.coe` format which has only one dimension. *States 2-6*

After loading the image and the kernel to their respective buffers, we inject these buffers directly into the PE grid, in one shot. If the buffers do not fill the PE grid, we inject the left-out empty PEs with zeroes. *State 7*

Post injection, we can get the partial sums. We write these partial sums into another BRAM, to store the results. Writing into the BRAM does not write the values into a designated `.coe` file. *States 8-11*

Now that PE-related steps are over, we move on displaying the function. To make sure the steps are over, we kept an index on *State 12* that returns to either *State 1* to move on to the next input, or continues on to *States 13-15*, where the result is read from the BRAM. We've also added hardware functionality here. Below, you can find a detailed figure with the whole FSM depicted in a flowchart.

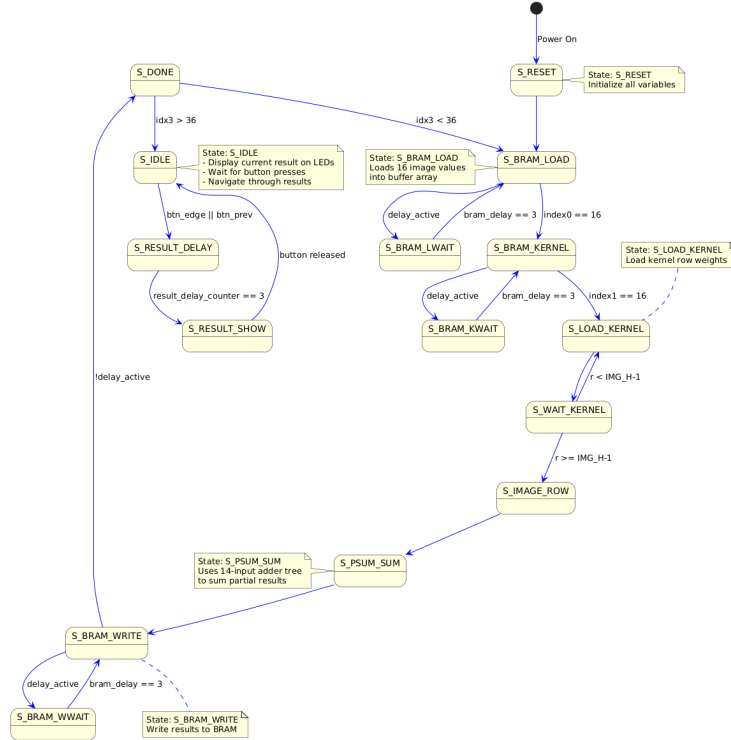


Figure 6: FSM Diagram for System Control

## 1.7 Hardware

For hardware, we've used the Nexys A7-T100 Development Board. To display the results, we've done the following:

- To show the output index (0-36 in test case), we've used the first set of Seven-Segment-Displays, highlighted with red.
- To show the partial sum in the particular index, we've utilized the second set of SSDs, as well as 16 LEDs situated beneath the two set of SSDs, highlighted with blue and purple respectively.
- To go through the output index and resetting, etc. we've used the on-board push buttons highlighted with green.

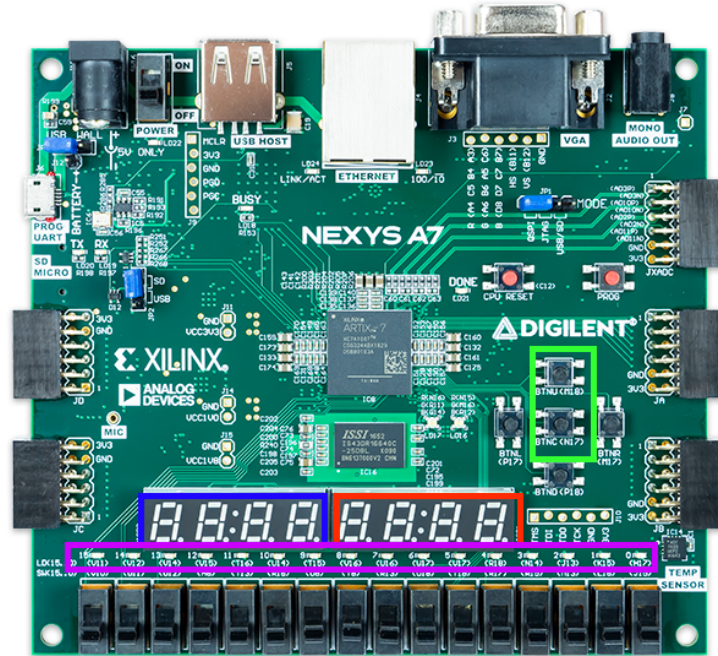


Figure 7: Hardware used for the project, with used peripherals highlighted with boxes.

Utilization wise, we have the following results. A total of:

- 1604 LUTs,
- 1937 FFs,



- 30 BRAM and 72 DSP blocks have been used.

With 1MHz clock, our WNS is around 968.9 ns. The bitstream is generated successfully, with no critical warnings nor errors present.

## References

- [1] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” in *2016 IEEE International Solid-State Circuits Conference (ISSCC)*, 2016, pp. 262–263.